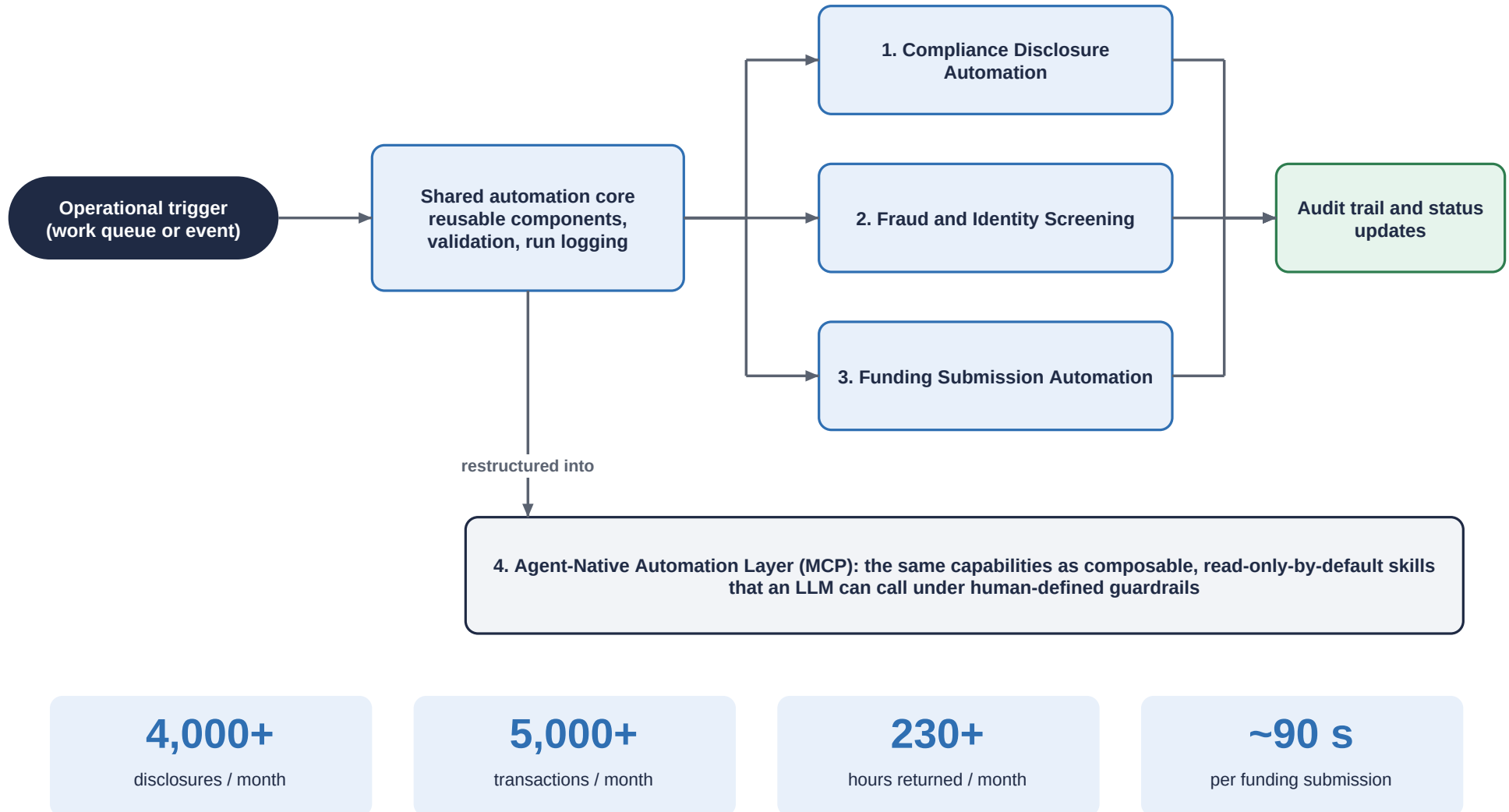


AI Ops Workflow: Process Flows

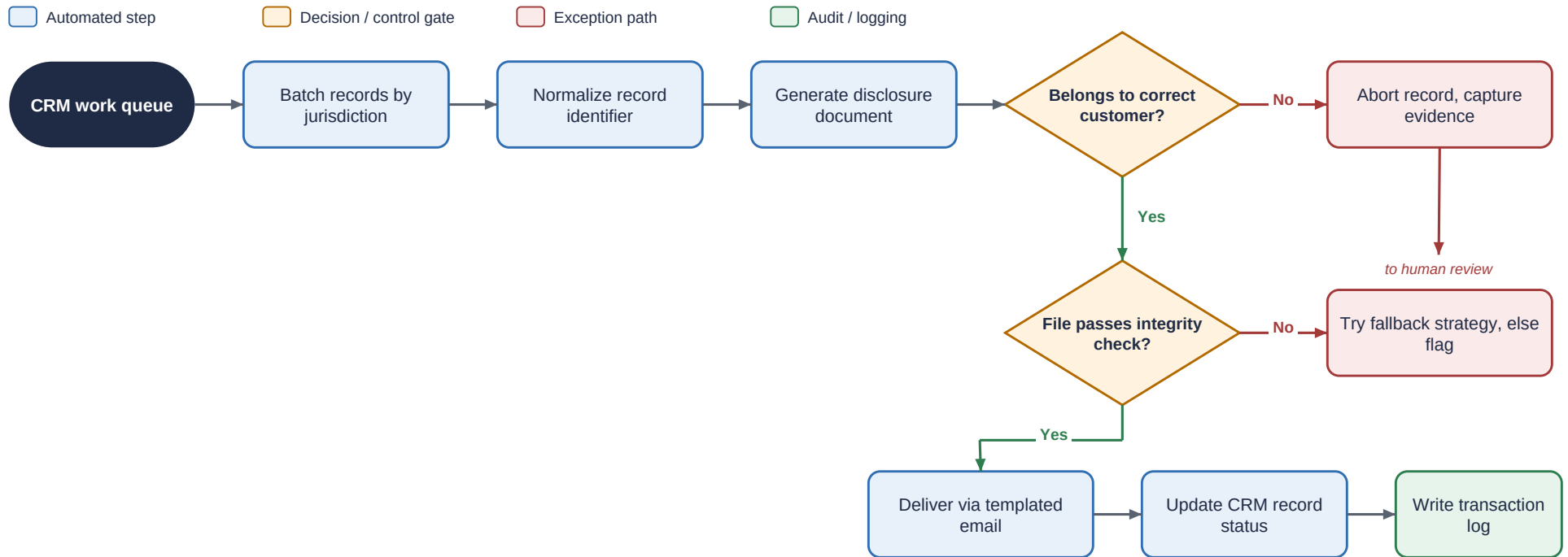
Automation for compliance, fraud screening, and funding in regulated financial services operations

End-to-end automations I designed, built, deployed, and supported. Each runs in a regulated workflow where a silent failure is a compliance event, so every flow below is built around control gates, exception routing, and an audit trail.



1. Compliance Disclosure Automation

Regulated credit disclosure delivery for every financed transaction, running unattended



Controls that made it production-grade

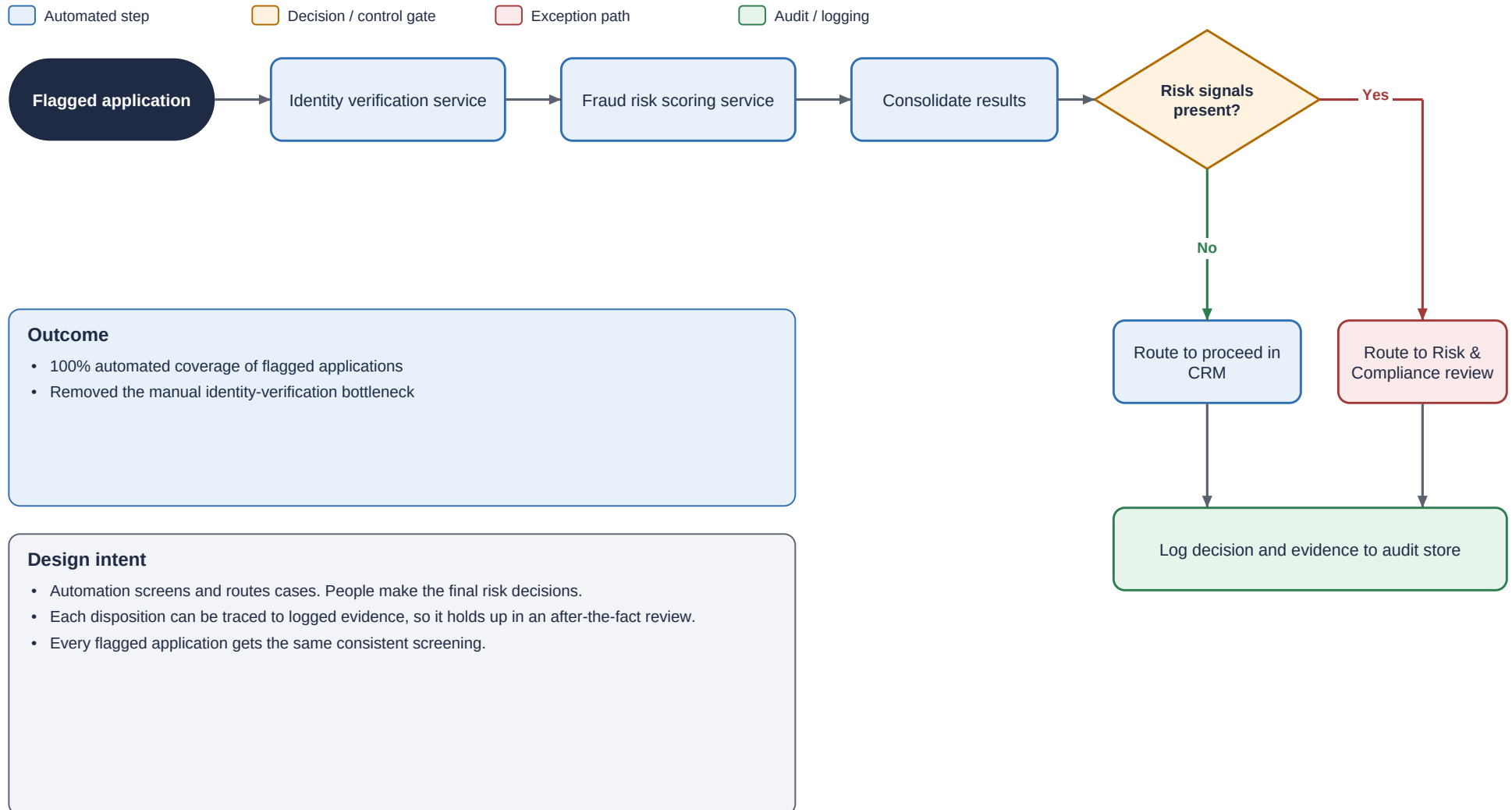
- Verification interlock: no document is delivered unless it is confirmed to belong to the right customer.
- Integrity gate: every file is validated before it can reach a customer.
- Logs are written even on failure, so every run leaves an audit trail.

Outcome

- 4,000+ disclosures automated per month
- 15+ minutes of manual work per customer down to ~2.5 minutes unattended
- Jurisdiction batching and identifier normalization removed most lookup failures

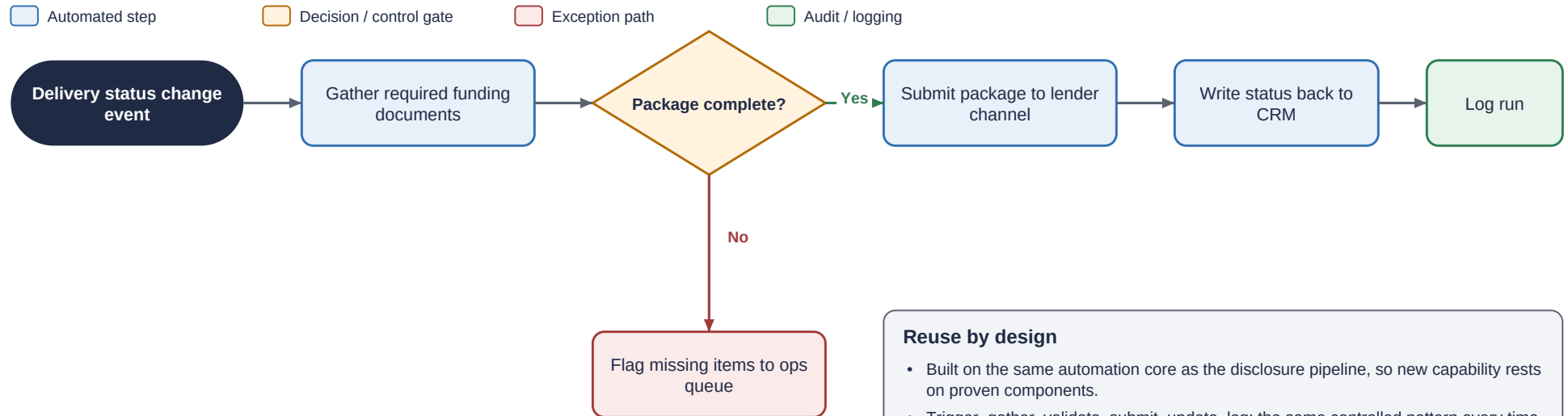
2. Fraud and Identity Screening

Automated identity verification and fraud risk screening, with every decision reconstructable



3. Funding Submission Automation

Event-driven submission of post-delivery lender funding packages



Reuse by design

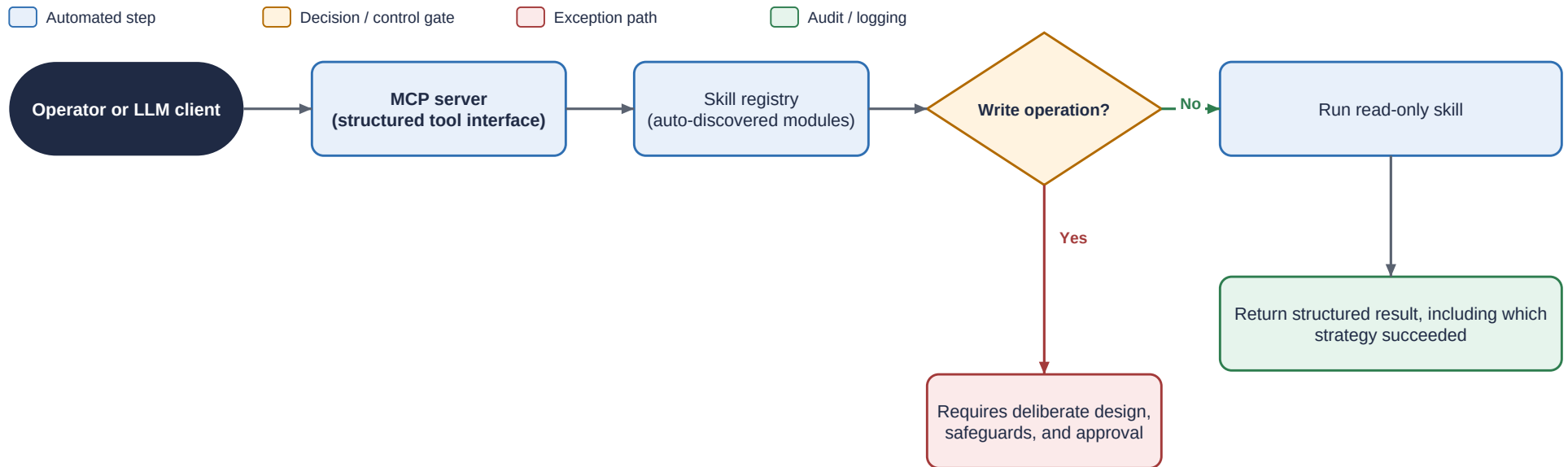
- Built on the same automation core as the disclosure pipeline, so new capability rests on proven components.
- Trigger, gather, validate, submit, update, log: the same controlled pattern every time.

Outcome

- ~90 seconds per transaction
- Weekly funding workload cut from 30+ hours to under 3
- Near-zero operational error rate

4. Agent-Native Automation Layer (MCP)

Restructuring fixed pipelines into composable skills an LLM can call, with guardrails built in



Why rebuild as an agent layer

- Pipelines run one fixed sequence. Adding each new capability cost as much as the last.
- Now a new capability is just a new module with a name, description, input schema, and run function.
- The same skills can be called from a terminal or an MCP client.

Engineering principles

- Fail loudly: a confusing error beats a clean-looking result that is quietly wrong.
- Always write the diagnostic artifact, even on failure.
- Read-only by default: write access is a decision, not a side effect.
- Documentation is a deliverable: symptom > root cause > fix > rule.